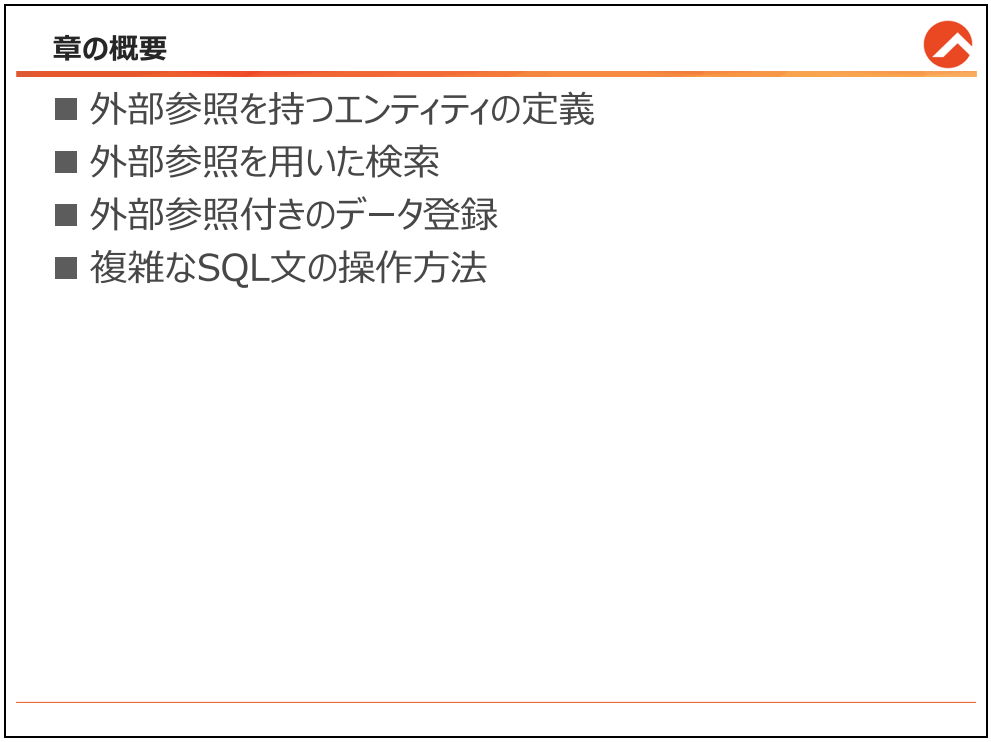
6

第6章　外部参照と複雑な SQL 操作

章の目標

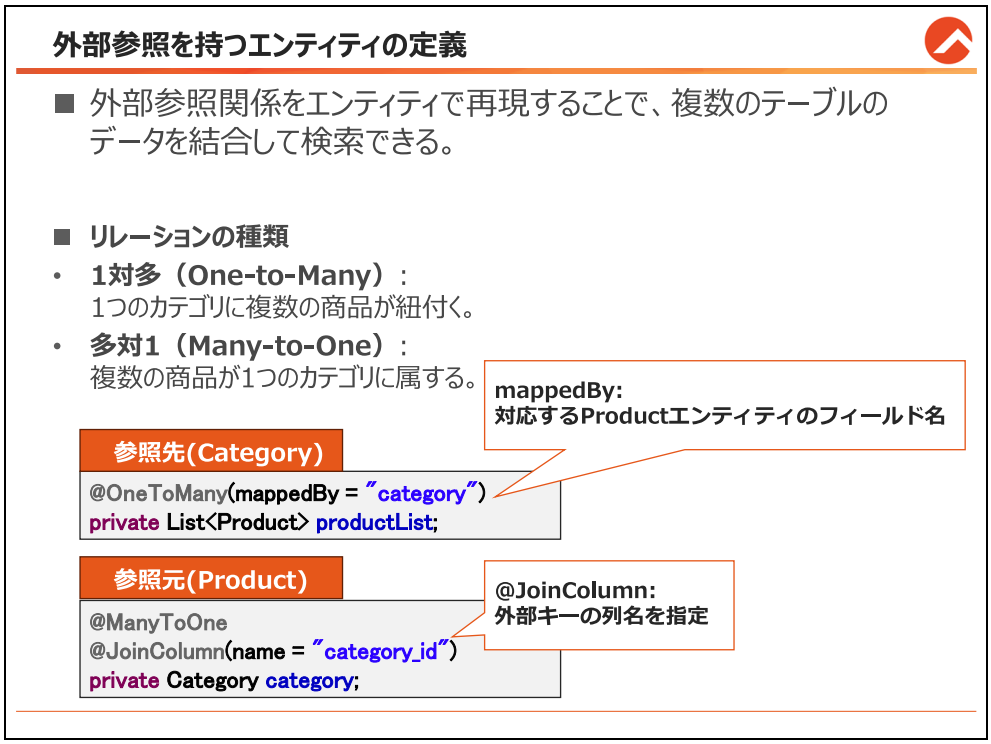
- 外部参照を持つエンティティを利用し、データベース間のリレーションシップを設定できる
- Spring Data JPA を用いた外部参照付きの検索や登録を実装できる
- 複雑な SQL クエリを実行し、実践的なデータ操作ができる

章の概要



本章では、外部参照を持つエンティティの操作方法について学びます。複数のテーブル間にリレーションシップがある場合のエンティティ定義や検索、登録、更新の方法を具体的に解説し、さらに複雑な SQL クエリをネイティブクエリで実装する方法を紹介します。Spring Data JPA の標準的な機能を基盤とし、実務に役立つデータ操作を学ぶことを目指します。

6.1. 外部参照を持つエンティティの定義



3 章では、1 つのテーブルに対応するエンティティを定義し、Spring Data JPA を用いた基本的な操作方法を学びました。本節では、**複数のテーブル間に外部参照の関係がある場合に、どのようにエンティティを設計し、リレーションシップを定義するのかを学びます。**具体的には、「商品(Product)」テーブルと「カテゴリ(Category)」テーブルのリレーションシップをエンティティとしてどのように表現するかを解説します。

【 作業手順①】新たなテーブルと、データの準備

以下のように、「 カテゴリ(Category) 」テーブルを新規作成し、既存の「 商品(Product) 」テーブルと外部参照で結合します。

※本書では、学習のためにデータベースが事前に構築されています。実務では、必要に応じてテーブルや外部キーの設定を行ってください。

＜Category テーブル＞

No	論理名称	物理名称	データ型	制約	備考
1	カテゴリ ID	id	INT	PRIMARY KEY	−
2	カテゴリ名	name	VARCHAR(30)	NOT NULL	−

<Category テーブルに格納されるデータ>

カテゴリ ID	カテゴリ名
1	テレビ・モニター
2	コンピュータ
3	冷蔵庫
4	洗濯機

このデータを基に、「 Product」テーブルにも新たに category_id 列を追加し、外部参照の関係を持たせる必要があります。

<既存の Product テーブルへの変更>

No	論理名称	物理名称	データ型	桁数	制約	備考
1	商品番号	id	INT	–	PRIMARY KEY AUTO_INCREMENT	自動採番
2	商品名	name	VARCHAR	30	–	–
3	価格	price	INT	–	–	–
4	カテゴリ ID	category_id	INT		FOREIGN KEY	(Category テーブルの id 列を参照)

この修正により、Product テーブルと Category テーブルが外部キーで結びつきます。

商品番号	商品名	価格	カテゴリ ID
1	スマートテレビ	30000	1
2	ゲーミングモニター	20000	1
3	ノートパソコン	50000	2
4	2 ドア冷蔵庫	50000	3
5	コンパクト冷蔵庫	15000	3
6	ワインクーラー	30000	3
7	全自動洗濯機	25000	4
8	乾燥機能付き洗濯機	40000	4
9	洗濯乾燥機	60000	4

カテゴリを管理する「Category」テーブルを追加し、外部参照を利用する形に拡張する準備が整いました。

外部参照とは

外部参照とは、1 つのテーブルが別のテーブルのデータを参照する関係を指します。データベースでは外部キー(Foreign Key)を使用してこの関係を表現します。

テーブル同士のリレーショナル(関係)が結ばれると、その関係性を表すために、以下の用語で区別します。

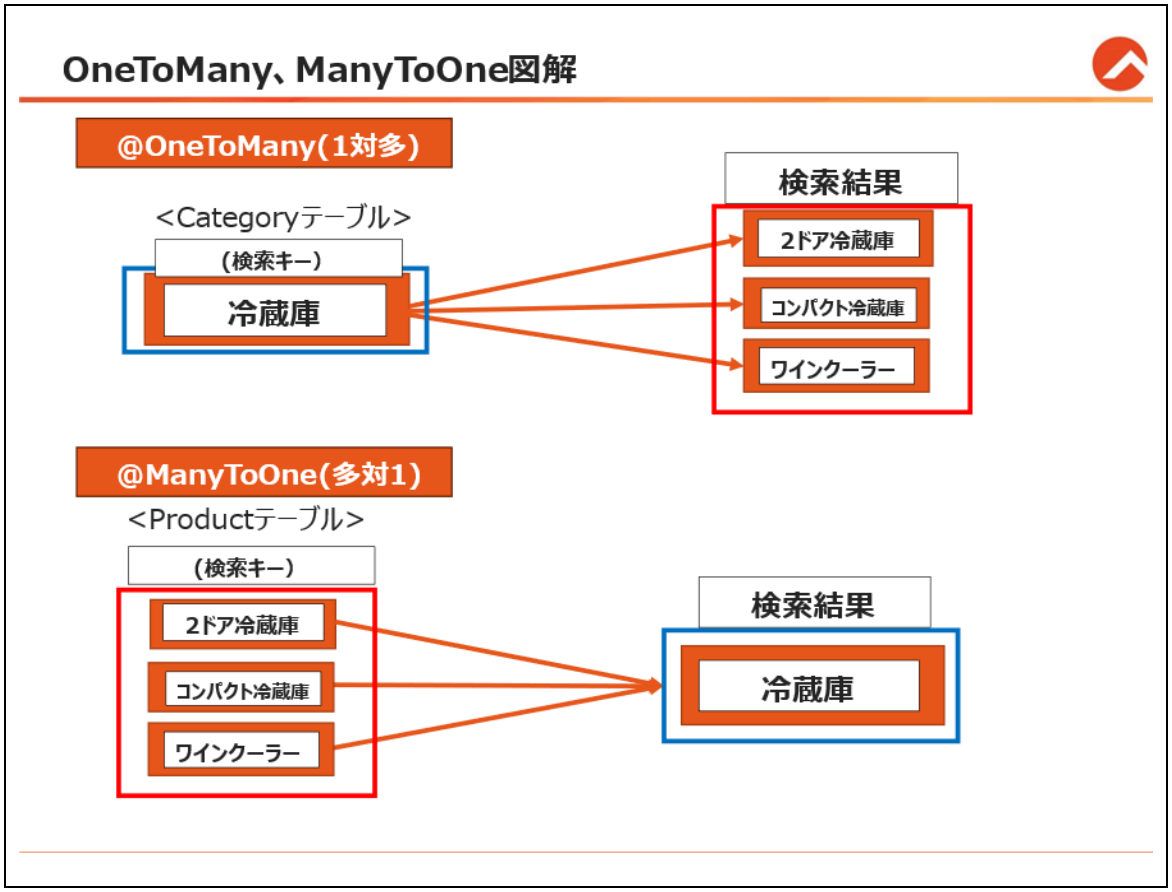
リレーションの種類

- 1 対多(One-to-Many)

  1 つのカテゴリ(例: 冷蔵庫)に対して複数の商品(例: 2 ドア冷蔵庫、ワインクーラー)が紐付く関係です。

- 多対 1(Many-to-One)

  複数の商品が 1 つのカテゴリに紐付く関係です。全ての商品は必ずいずれかのカテゴリに属します。



Category テーブルからの視点では、Product テーブルとの関係は、1 対多と表現できます。

Product テーブルからの視点では、Category テーブルとの関係は、多対 1 と表現できます。

【 作業手順②】参照先エンティティの作成

では、テーブルとの関係性を、Java の世界にエンティティとして、忠実に再現しましょう。
まずは、category テーブルに対応するエンティティ Category を作成します。

…/entity/Category.java

```
package jp.co.trainocate.answer.entity;

import jakarta.persistence.OneToMany;

@Entity
@Data
@Table(name = "category")
public class Category {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @Column(length = 30)
    private String name;

    @OneToMany(mappedBy = "category")
    private List<Product> productList;
}
```

上記の Category エンティティは、category テーブルに対応しています。各フィールドはテーブル内の列に対応しており、特に id フィールドは主キーとして定義されています。

また、**productList フィールド**は 後述する Product エンティティを格納するリストで、**一対多の関係を作るために複数のデータを格納する必要がある**ため、必須です。

@OneToMany アノテーションは、1 つのカテゴリが複数の商品を持つ「1 対多」の関係を表します。たとえば、「冷蔵庫」というカテゴリに、「2 ドア冷蔵庫、コンパクト冷蔵庫、ワインクーラー」が該当するパターンです。

@OneToMany アノテーションには **mappedBy 属性**があります。この属性は、後述する Product エンティティに定義されるフィール名を記載します。Product エンティティの説明の後に、再度ご確認ください。

【 作業手順③】参照元エンティティの修正

次に、Product テーブルに対応するエンティティ Product を修正します。Product テーブルは category テーブルを外部参照しているため、Product エンティティには Category エンティティをフィールドとして含めます。

…/entity/Product.java

```
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;

@Entity
@Data
@Table(name = "product")

public class Product {

        @Id
        @GeneratedValue(strategy = GenerationType.IDENTITY)
        private Integer id;

        @Column(length = 30, nullable = false)
        private String name;

        @Column(nullable = false)
        private Integer price;

        @ManyToOne
        @JoinColumn(name = "category_id")
        private Category category;
}
```

最後の category フィールドにご注目ください。今まではテーブル上の列名がエンティティのフィールド名に合わせる必要がありました。よって、以下のようなテーブル構成なら、「 Integer categoryId」と定義するべきでしょう。

4	カテゴリ ID	category_id	INT		FOREIGN KEY	(Category テーブルの id 列を参照)

しかし、category_id ではなく、「 Category category」 のフィールドを定義しています。

JPA では、外部キー(例: category_id 列)を直接操作せず、リレーション先のエンティティ(例: Category)をフィールドとして扱います。これにより、オブジェクト指向の設計とデータベース操作を統一できます。

```
@ManyToOne
@JoinColumn(name = "category_id", nullable = false)
private Category category;
```

このように、**外部参照の関係を持つテーブルのエンティティを作成する際には、列名ではなく、参照先のエンティティをフィールドとして定義します。**Product テーブルは Category テーブルと外部参照の関係にあるため、Product エンティティ内に Category 型のフィールドを追加しています。

そして、**@ManyToOne アノテーションは、複数のアイテムが 1 つのカテゴリに属する「多対 1」の関係を表します**。Product エンティティの category フィールドに付与されています。

たとえば、「2 ドア冷蔵庫、コンパクト冷蔵庫、ワインクーラー」のようなデータは、「冷蔵庫」のカテゴリに属しているということです。

@JoinColumn アノテーションは、参照元テーブルのどの列と参照先テーブルのどの列が外部参照制約の関係にあるかを定義します。 **name 属性には、外部キーとして使用するテーブル列名(例: category_id)を指定します。**

このアノテーションを記述すると、**検索時に参照関係にあるテーブルが内部結合され、自動的にデータを取得**できます。つまり、**Product テーブルを対象に検索したら、Category テーブルのデータも取得できるようになる**ということです。なお、参照関係にあるテーブルのレコードはエンティティのフィールドに代入されます。

これにより、テーブル間のリレーションシップが Spring 上で明確に定義され、データベース操作が容易になります。

【 作業手順④】 CategoryRepository の作成

…/repository/CategoryRepository.java

```
package jp.co.trainocate.answer.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import jp.co.trainocate.answer.entity.Category;

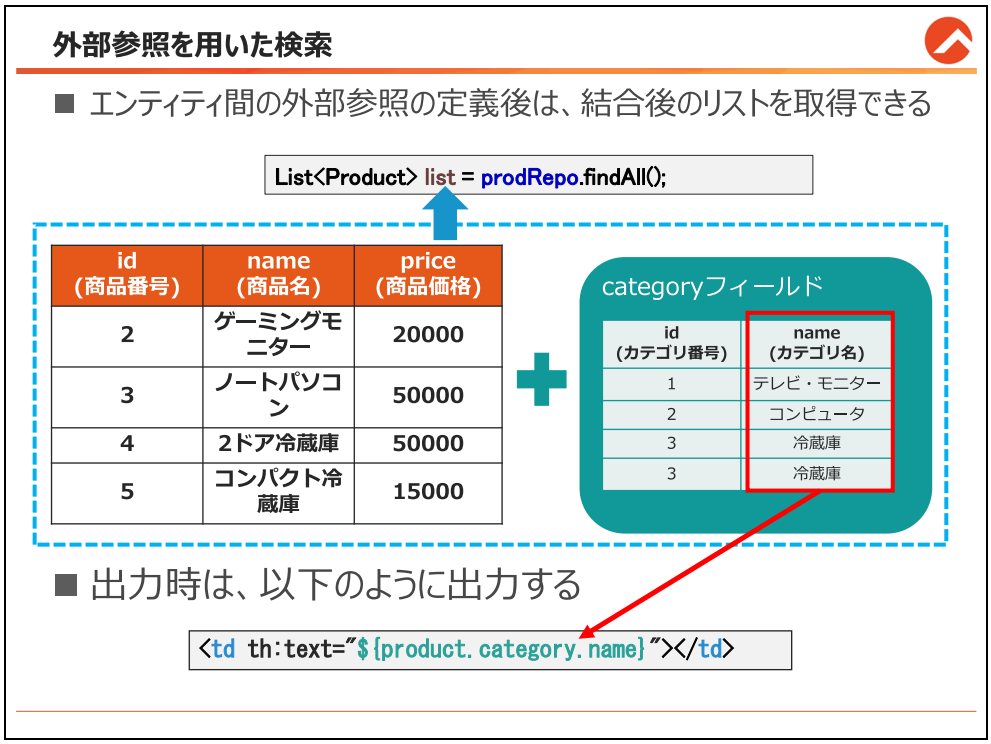
public interface CategoryRepository extends JpaRepository<Category,
Integer> {

}
```

Category の検索も後ほど必要となりますので、リポジトリを作成します。

「Category」は、別 API と名前が被ることがありますので、今後、Import の際は、注意してください。

6.2. 外部参照を用いた検索



本節では、Spring Data JPA を用いて、外部参照を持つテーブル間の基本的な検索を実装する方法を学びます。外部参照を定義したエンティティを活用し、関連データを簡単に取得する基礎的な操作に重点を置きます。

【 作業手順①】商品コントローラーの再確認

以下は、3 章で作成した ProductController とほぼ同じ内容です。ただし、外部参照をエンティティに定義したため、挙動に変化があります。

…/controller/ProductController.java

```
@Controller
@RequestMapping("/product")
public class ProductController {
	@Autowired
	private ProductRepository prodRepo;

	@GetMapping("/findAll")
	public String showProductList(Model model) {
		List<Product> list = prodRepo.findAll();
	  //System.out.println(list); //外部参照の定義後は、エラーになる
		model.addAttribute("products", list);
		return "product/list";
	}
```

■ findAll メソッドの結果

外部参照を定義した後、findAll メソッドの実行結果には、**Product のデータに加えて、関連する Category のデータも含まれる**ようになります。

一覧取得時に、関連するカテゴリ情報も自動的に取得されます。

■ 標準出力の注意点

外部参照を持つエンティティは構造が複雑になります。特に、エンティティ間の双方向参照が設定されている場合、エンティティのインスタンスを System.out.println で出力すると、スタックオーバーフローエラーが発生する可能性があります。

Product が Category を参照し、Category が Product を再参照するためです。

【 作業手順②】View にカテゴリ名を出力するよう修正

商品データをリスト表示する際に、カテゴリ名も合わせて表示するように list.html を修正します。以下のコードは、商品リストを HTML テーブルとして表示するサンプルです。
変更点として、**カテゴリ名を表示する列を追加し、product.category.name でカテゴリ名を取得**しました。

…/resources/templates/product/list.html

```
<h1>商品一覧</h1>
        <table>
                <tr>
                        <th>商品番号</th>
                        <th>商品名</th>
                        <th>価格</th>
                        <th>カテゴリ</th>
                </tr>
                <tr th:each="product: ${products}">
                        <td th:text="${product.id}"></td>
                        <td th:text="${product.name}"></td>
                        <td th:text="${product.price}"></td>
                        <td th:text="${product.category.name}"></td>
                </tr>
        </table>
```

カテゴリ名の取得

<td th:text="${product.category.name}"></td>の部分で、Product エンティティに紐付く Category エンティティの name フィールドを表示できます。

一覧取得時に、関連するカテゴリ情報も自動的に取得されます。product.category は Product エンティティの@ManyToOne で定義されたリレーションを利用して、Category エンティティを取得できます。

このように、外部参照が正しく定義されていると、対象のエンティティだけではなく、紐づいている別エンティティのデータ群も取得可能です。

【参考】null データの参照時のエラー対策

外部参照が定義されたエンティティでは、関連するデータ(ここではカテゴリ)が設定されていない場合に注意が必要です。具体的には、**product.category が null の場合に product.category.name を参照しようとすると、エラーが発生します。**

このような場合、カテゴリが未設定(null)でもエラーを回避し、適切に表示できるように処理を追加します。

① th:if 文を利用

5 章で学習したとおり、Thymeleaf では、th:if 属性を利用して条件を設定することができます。

```
<td th:if="${product.category != null}" th:text="${product.category.name}"></td>
<td th:unless="${product.category != null}">未設定</td>
```

■ **th:if="${product.category != null}"**
この部分は、「カテゴリが設定されている場合」に実行されます。カテゴリ名を表示します。

■ **th:unless="${product.category != null}"**
この部分は、「カテゴリが未設定の場合」に実行されます。「未設定」という文字列を表示します。

② 条件式を 1 行で記述する方法(三項演算子)

上記のように複数行で条件を記述する代わりに、条件式を使って 1 行で記述する方法もあります。

```
<td th:text="${product.category != null ? product.category.name : '未設定'}"></td>
```

■ **product.category != null:**

「カテゴリが設定されているか」をチェックします。

■ **?(三項演算子):**

条件が true の場合に左側の値(product.category.name)を表示します。

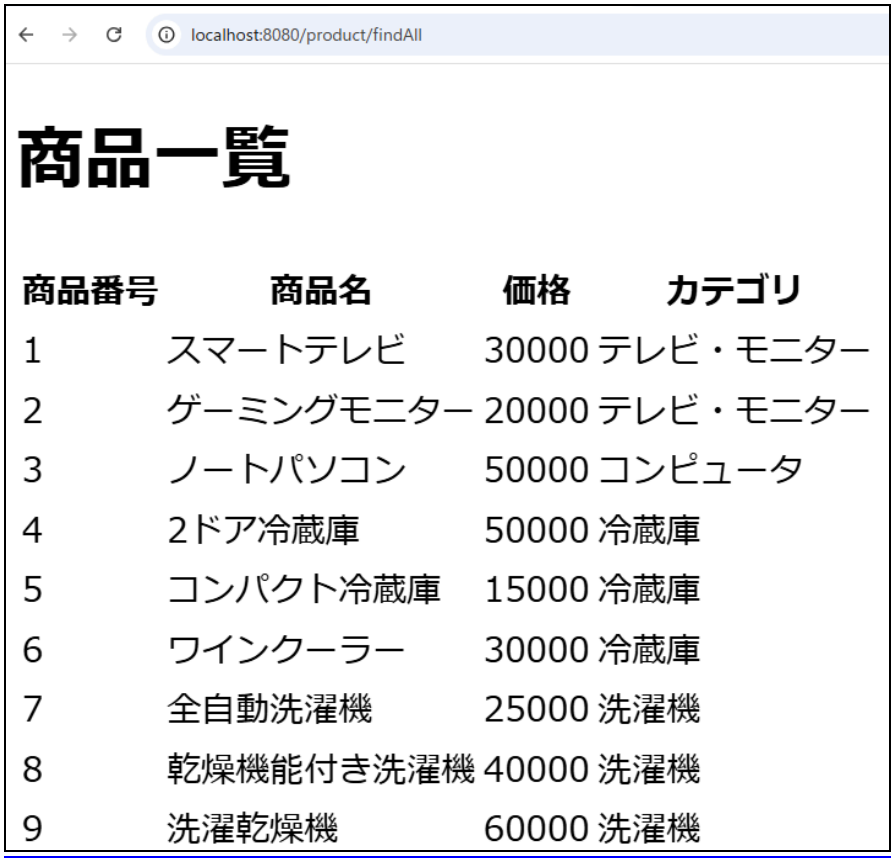
■ **:(else 部分):**

条件が false の場合に右側の値('未設定')を表示します。

動作確認

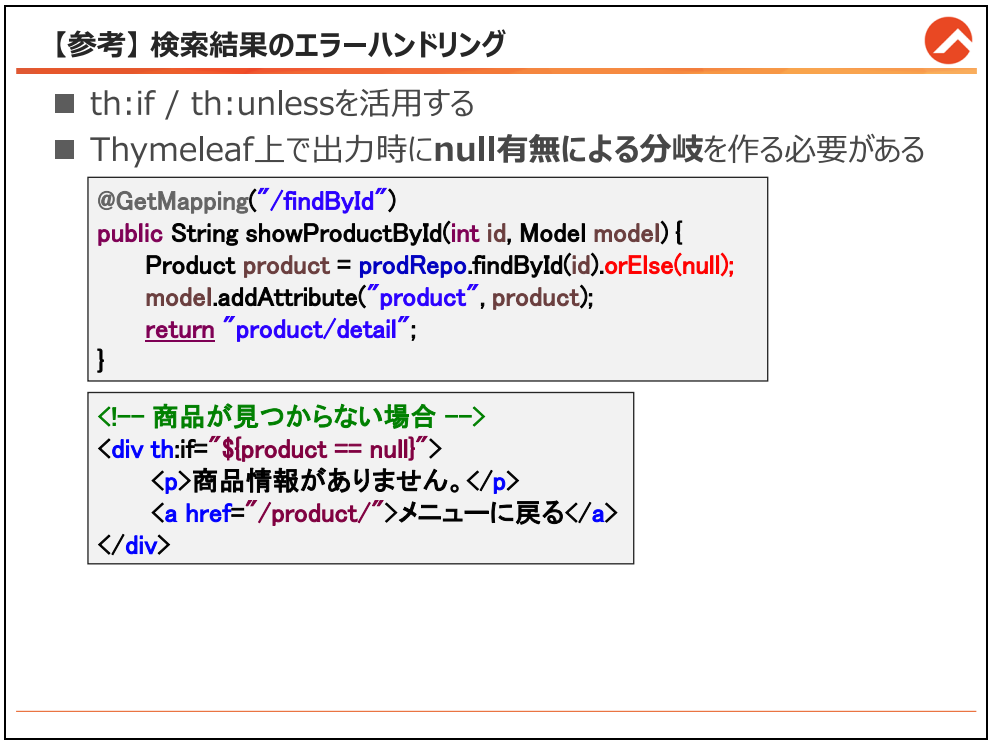
http://localhost:8080/product/findAll

3 章と同様に、Controller の URL マッピングが、[/product]
メソッドの URL マッピングが[/findAll] になっているため、このような URL を指定します。



カテゴリ列に商品ごとのカテゴリ名が結合され表示されることを確認できます。

6.2.1.【 参考】検索結果のエラーハンドリング



本節では、検索時のエラーハンドリングについて学びます。存在しない ID で検索した場合に発生するエラーを回避し、より安全にデータを取得する方法について学びます。また、カテゴリ名を表示することで、リレーションの復習も行います。

【 作業手順①】findById(id).orElse(null)の確認

…/controller/ProductController.java

```
//省略

@GetMapping("/findById")
	public String showProductById(int id, Model model) {
		Product product = prodRepo.findById(id).orElse(null);
		model.addAttribute("product", product);
		return "product/detail";
	}
```

findById(id).orElse(null)を使用すると、データが存在しない場合に null が返され、安全に処理を進めることができます。

もちろん、エンティティ間の定義が完了しているので、**findById でも結合した結果を取得**できます。

【 作業手順②】商品が見つからなかった際の表示の切り替え

…/resources/templates/product/detail.html

```
<h1>商品詳細</h1>

    <!-- 商品が見つからない場合 -->
    <div th:if="${product == null}">
        <p>商品情報がありません。</p>
        <a href="/product/">検索メニューに戻る</a>
    </div>

    <!-- 商品が見つかった場合 -->
    <div th:unless="${product == null}">
        <table>
            <tr>
                <th>商品番号</th>
                <th>商品名</th>
                <th>価格</th>
                <th>カテゴリ</th>
            </tr>
            <tr>
                <td th:text="${product.id}"></td>
                <td th:text="${product.name}"></td>
                <td th:text="${product.price}"></td>
                <td th:text="${product.category.name}"></td>
            </tr>
        </table>

    <!--...省略-->

    </div>
```

で囲まれた部分は、Controller で Model に格納されたデータの有無に応じて、表示内容を切り替える処理を行っています。

このように、**複数行の Thymeleaf 条件式は、視認性や保守性を高めるために を使用することを推奨します。**

※単体のデータの有無ではなく、リスト内の判定については、5.1.2 の②を再度ご確認ください。

```
<div th:if="${#lists.isEmpty(products)}">
    商品情報がありません。
</div>
<div th:unless="${#lists.isEmpty(products)}">
    <!-- 商品情報リストの内容(<table>～</table>)をここに記載 -->
</div>
```

動作確認

http://localhost:8080/product/

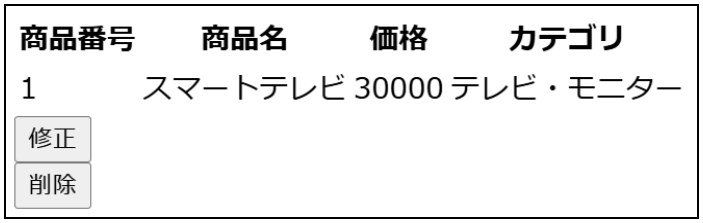
① ID 検索フォームに、存在しない商品番号を入れ、検索します。



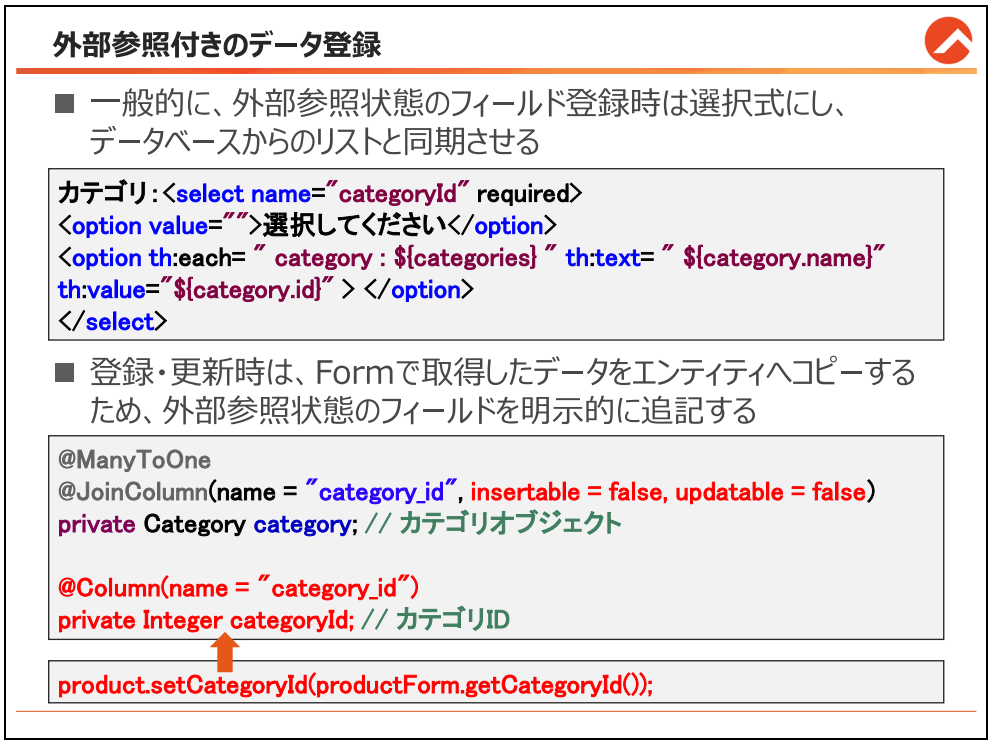
② 例外のエラーページの代わりに、商品情報なしのメッセージが表示されることを確認してください。



③ 検索ページに戻り、今度は「 1」で商品番号検索を行い、カテゴリ名を含めた商品データが表示されるかを確認してください。



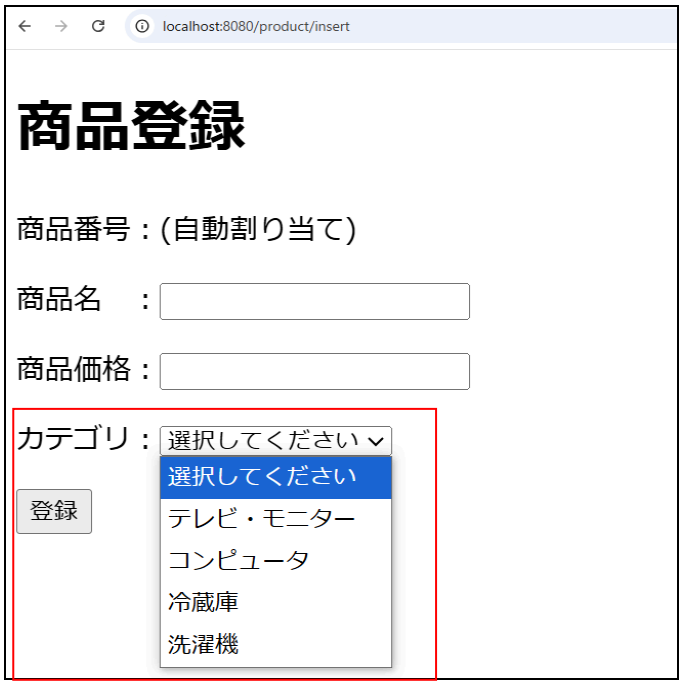
6.3. 外部参照付きのデータ登録



本節では、商品データとカテゴリデータのリレーションを考慮したデータ登録を実装します。特に、商品情報を登録する際にカテゴリを選択する方法を学びます。

また、フォームから送信されたカテゴリ ID を商品エンティティに関連付けて登録する際に直面する課題を説明し、その解決方法を実装します。

商品登録に選択式のカテゴリを追加する時



今回は、商品データにカテゴリを登録する機能を追加します。
カテゴリのように、多対 1 の外部参照関係を持つデータは、Web 上では選択式(セレクトメニュー) で登録するのが一般的です。

以下は、固定値を用いたセレクトメニューの例です。

…/resources/templates/product/insert.html

```
<h1>商品登録</h1>
        <form th:action = "@{/product/insert}" method = "POST">
        <p>商品番号：(自動割り当て)</p>
        <p>商品名　：<input type = "text" name = "name"></p>
        <p>商品価格：<input type = "number" name = "price"></p>
        <p>カテゴリ：
                <select name="categoryId" required>
                        <option value = "">選択してください</option>
                        <option value = "1">テレビ・モニター</option>
                        <option value = "2">コンピュータ</option>
                        <option value = "3">冷蔵庫</option>
                        <option value = "4">洗濯機</option>
                </select>
        </p>
```

このように固定値を記載する方法は簡単ですが、以下のような問題点があります。

- **新しいカテゴリの追加や既存カテゴリの修正が手間**
  HTML コードを手動で変更する必要があるため、管理が煩雑になります。
- **データベースとの不一致**
  UI とデータベースが同期しなくなる可能性があり、意図しない不具合が発生します。

これらの問題を解決するために、データベースからカテゴリデータを取得し、動的にセレクトメニューを生成する方法を採用します。

【 作業手順①】 登録処理の準備 – ProductForm の修正

登録画面のフォームでは以下のデータが送信されます。

- 商品名(name)
- 商品価格(price)
- カテゴリ ID(categoryId)

新たに追加された categoryId をコントローラで受け取るため、ProductForm クラスを修正します。

…/form/ProductForm.java

```
@Data
public class ProductForm {
        private Integer id; //商品番号
        private String name; //商品名
        private Integer price; //商品価格

        private Integer categoryId; //カテゴリ ID
}
```

送信される際に、select 要素の name 属性値が、categoryId ですので、Integer categoryId と定義します。

【 作業手順②】 登録画面遷移用のコントローラーの修正

…/controller/ProductController.java

```
import jp.co.trainocate.answer.entity.Category;
import jp.co.trainocate.answer.repository.CategoryRepository;

@Controller
@RequestMapping("/product")
public class ProductController {
	@Autowired
	private ProductRepository prodRepo;
	@Autowired
	private CategoryRepository cateRepo;

//検索系メソッドは省略

@GetMapping("/insert")
	public String showProductInsertView(Model model) {
		// カテゴリリストを取得
		 List<Category> list = cateRepo.findAll();
		 model.addAttribute("categories", list);
		 return "product/insert";  // 登録入力画面へ
	}
```

次は、Controller のハンドラメソッドを修正します。元々、@GetMapping(“/insert”)のメソッドは、今まで単純に登録をする入力画面へ遷移するだけの単純なコードでした。

ただ、<option>にカテゴリデータを動的に出力するには、コントローラで事前にモデルにカテゴリリストを格納して送信しておく必要があります。

コードの解説:

- **CategoryRepository の依存性注入:**

  カテゴリデータをデータベースから取得するため、CategoryRepository を @Autowired で注入します。

- **findAll()による全件取得:**

  カテゴリリストを findAll()メソッドで取得します。

- **model.addAttribute:**

  カテゴリリストを **categories** という名前でモデルに格納し、ビューで利用可能にします。

- **入力画面テンプレートへの遷移:**

  return "product/insert";で商品登録画面へ遷移します。

これで、登録画面へカテゴリのリストの送信ができましたので、登録画面の View を実装しましょう。

【 作業手順③】 カテゴリのリストの動的表示

```
<p>カテゴリ：<select name="categoryId" required>
        <option value="">選択してください</option>
        <option th:each="category : ${categories}"
                th:text="${category.name}"
                th:value="${category.id}" >
        </option>
    </select>
    </p>
```

修正点の説明:

- th:each:
  データベースから取得したカテゴリリスト(categories)をループ処理で展開します。
- th:text:
  メニューに表示するカテゴリ名(category.name)を動的に設定します。
- th:value:
  フォーム送信時にカテゴリ ID(category.id)をコントローラに渡します。登録処理ではこの ID を使用します。

ポイント:

- データベースのカテゴリデータに変更があっても、フォームは自動的に更新されます。
- セレクトメニューの<option>タグがデータベースの内容と常に同期します。

【 作業手順④】登録処理の実装 － 不具合の確認

ProductForm に categoryId フィールドを追加したことで、登録処理用のメソッドでカテゴリ ID を受け取ることができるようになりました。次に、ProductForm のデータを Product エンティティにコピーして保存する処理を実装します。

…/controller/ProductController.java

```
@PostMapping("/insert")
public String insertProduct(ProductForm productForm) {
	Product product = new Product();
	product.setName(productForm.getName());
	product.setPrice(productForm.getPrice());

	// カテゴリ ID を設定する処理を追加しようとする
	product.setCategoryId(productForm.getCategoryId()); // エラー発生

	prodRepo.save(product);
	return "redirect:/product/findAll";
}
```

3 章で学習した通り、prodRepo.save(product)をする前には、ProductForm のデータを Product エンティティにコピーしておく必要がありました。

しかし、いざ product.setCategoryId( ) メソッドを呼び出そうとしたら、そもそも**Product エンティティには、categoryId ではなく、Category category でフィールドを定義していたため、setCategoryId メソッドが生成されていません。**よって、**エラー**になります。

```
@Entity
@Data
@Table(name = "product")

public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @Column(length = 30, nullable = false)
    private String name;

    @Column(nullable = false)
    private Integer price;

    @ManyToOne
    @JoinColumn(name = "category_id")
    private Category category; //categoryIdではない

}
```

【 作業手順⑤】Product エンティティに categoryId フィールドの追加

作業手順④の不具合を解決するため、**Product エンティティに categoryId フィールドを追加**します。
ただし、categoryId と category の 2 つのフィールドが同じ列を参照するため、重複マッピングに注意が必要です。

…/entity/Product.java

```
// 上記省略
  @Column(nullable = false)
  private Integer price;

  @ManyToOne
  @JoinColumn(name = "category_id", insertable = false, updatable = false)
  private Category category; // カテゴリオブジェクト

  @Column(name = "category_id")
  private Integer categoryId; // カテゴリ ID
}
```

フィールド追加のポイント

- **@Column(name = "category_id")を明示的に指定する理由:**

  データベースの category_id 列を categoryId フィールドにマッピングするために必要です。
  指定しないと、フィールド名を基に自動推論され、列重複によるエラーが出ます。

- **insertable = false, updatable = false を設定する理由:**

  @ManyToOne の category フィールドと@Column の categoryId が同じカラムを参照していると、**登録や更新時にどちらを優先するか Hibernate が判断できず**、以下のようなエラーが発生します

  > Repeated column in mapping for entity: Product column: category_id (should be mapped with insert="false" update="false")

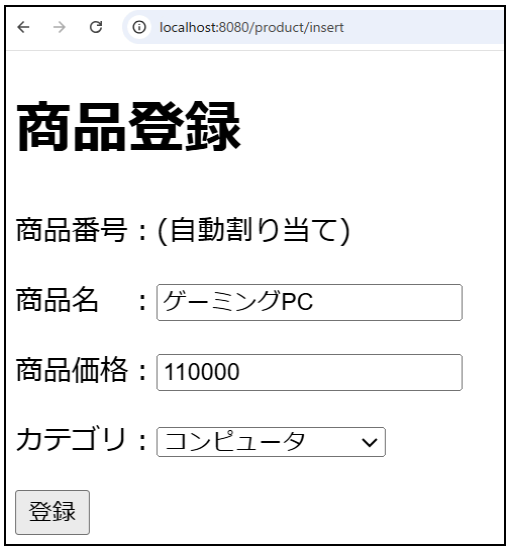
  insertable = false, updatable = false を設定することで、**category フィールドが登録や更新に関与しないようにし、categoryId だけを使用するように制御**できます。

Product エンティティの修正後、保存すると、登録処理用メソッドのエラーが消え、正常にカテゴリ ID を登録できるようになります。

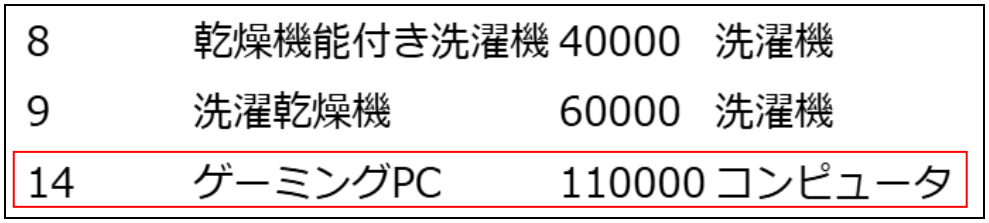
動作確認

http://localhost:8080/product/insert

① 任意のデータを入力・選択し、「登録」ボタンをクリックします。

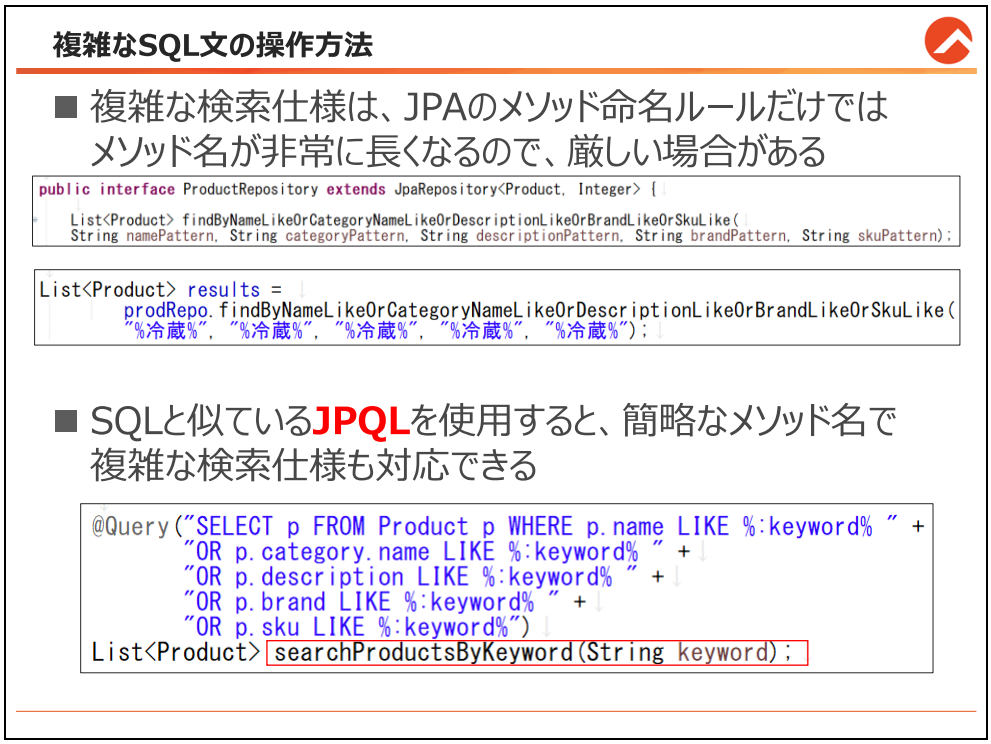


② 商品一覧に、カテゴリを含む新規データが追加されたら OK です。



※この節では、登録処理のみ説明しましたが、更新処理も同様に categoryId を受け取り、エンティティにコピーする 1 行さえ追記すれば、実装できますので参考にしてください。

6.4. 複雑な SQL 文の操作方法



これまでの章では、Spring Data JPA のリポジトリメソッド名を利用してクエリを実行する方法を学びました。しかし、複雑な SQL 文を作成する場合、リポジトリメソッド名が非常に長くなり、可読性や保守性が低下するという問題があります。

本節では、JPA の JPQL(Java Persistence Query Language)を使用して、クエリを記述する方法を学びます。JPQL は SQL に似た構文を持ちつつ、Java のエンティティを操作するために最適化されています。

JPQL の基本構文を学び、簡単な例を通じて理解を深めます。最後に【 参考】として、複雑な仕様に基づいた JPQL の実装例を紹介します。

6.4.1. 複雑な検索をメソッド名で定義する際のデメリット

実務では、テーブル内の列が増えたり、複雑な仕様の検索をしないといけない場面がよくあります。そのような状況で、Spring Data JPA のメソッド命名規則を使用して複雑な検索を実現しようとすると、いくつかのデメリットが発生します。

以下の例を基に説明します。

＜例: 拡張された Product テーブル＞

以下のように、Product テーブルが拡張されている状態を仮定します。

列の物理名	列の論理名
id	商品番号
name	商品名
price	価格
description	商品説明
brand	ブランド名
sku	SKU コード
category_id	カテゴリ ID
price	価格

そして、以下のような検索機能を実装する必要があったと仮定します。

- **例題: 商品とカテゴリを検索する仕様**

  「商品名」「カテゴリ名」「商品説明」「ブランド名」「SKU コード」の**いずれかに部分一致する商品を検索**し、該当する商品一覧を取得します。

- **出力形式:**

  商品番号、商品名、カテゴリ名、商品説明、ブランド名、SKU コード、価格

Spring Data JPA のメソッド命名規則で再現する場合

3 章で学習した通り、Spring Data JPA では findBy メソッド名に条件を列挙してクエリを作成できます。しかし、この仕様を満たすために適用すると、以下のような非常に長いメソッド名になってしまいます。

<ProductRepository に定義>

```
public interface ProductRepository extends JpaRepository<Product, Integer>{

List<Product> findByNameLikeOrCategoryNameLikeOrDescriptionLikeOrBrandLikeOrSkuLike(
  String namePattern, String categoryPattern, String descriptionPattern, String
brandPattern, String skuPattern);
```

以下のように、全ての列に対して同じ検索キーワードを適用する場合でも、それぞれ個別のパラメータを渡す必要があります。

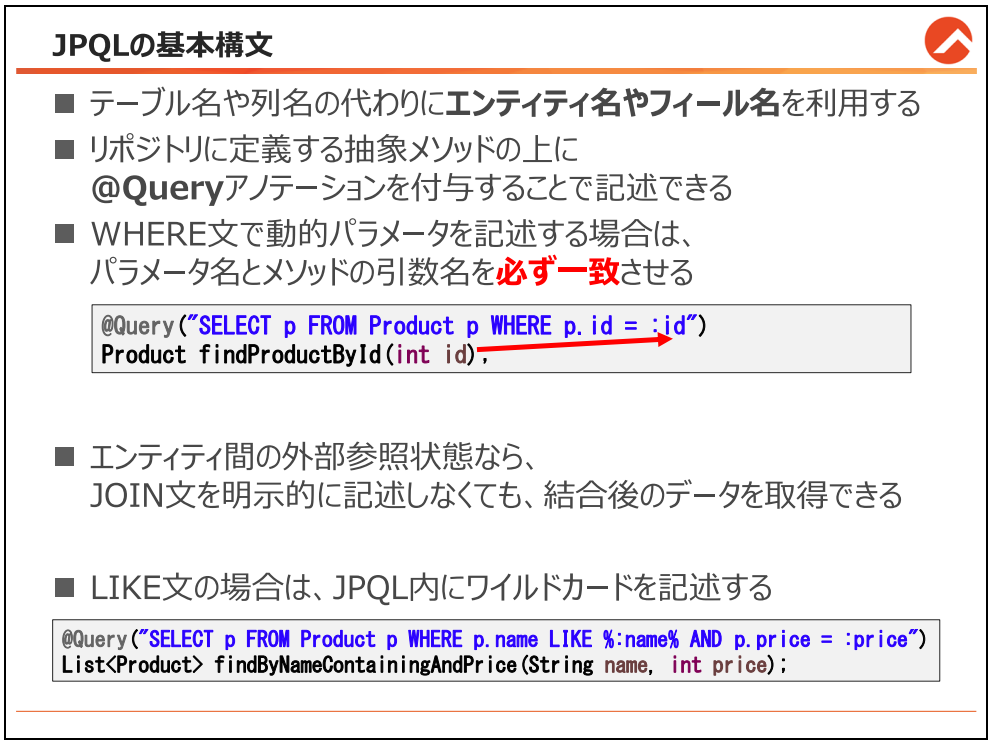
＜ProductController から、メソッドを呼び出す時＞

```
List<Product> results =
productRepository.findByNameLikeOrCategoryNameLikeOrDescriptionLikeOrBrandLikeOrSkuLike(
    "%冷蔵%", "%冷蔵%", "%冷蔵%", "%冷蔵%", "%冷蔵%");
```

非常に長いメソッド名と、冗長なパラメータを渡す必要があるため、JPA のメソッド命名規則だと、厳しいです。このままだと、可読性はもちろん、保守性も低い実装になってしまいます。

このような**複雑な検索条件を効率的に実装するには、SQL 文を直接記述する方法が適しています。**次節では、Spring Data JPA の JPQL と、@Query アノテーションを使用して SQL 文を直接記述する方法について説明します。

6.4.2. JPQL の基本構文



JPQL の特徴

JPQL(Java Persistence Query Language)は、JPA エンティティを基に操作を行うクエリ言語です。

SQLと似た構文ですが、**テーブル名や列名ではなく、エンティティ名やフィールド名を使用**します。

JPQL の主な特徴:

- **エンティティモデルに基づいてクエリを記述**
  データベースのテーブル構造を意識することなく、Java のオブジェクト指向に沿ったクエリが記述できます。
- **リポジトリに直接記述可能**
  クエリはリポジトリインターフェースに記述しますので、以前の章で学んだ感覚のまま実装できます。
- **メソッド名の命名規則が不要**
  JPQL を使用する場合、自由にメソッド名を設定できます。

全件検索の実装

JPQL の基本を学ぶため、すべての商品データを取得するクエリを JPQL で実装します。以下のようにリポジトリインターフェースにクエリを記述します。

…/repository/ProductRepository.java

```
public interface ProductRepository extends JpaRepository<Product, Integer>{

        @Query("SELECT p FROM Product p")
         List<Product> findAllProducts();

        //他の抽象メソッドは省略
}
```

- **findAllProducts**
  メソッド名は自由に決められます。今回は分かりやすい名前にしています。引数と戻り値を開発者が決めることは、前の章と同じです。全件検索ですので、引数無しで戻り値は List<Product>で定義しています。
- **@Query アノテーションの付与**
  JPQL を直接文字列として、@Query アノテーションの引数に記述できます。
- **SELECT p FROM Product p**
  エンティティ Product をすべて取得するクエリです。p は表別名で、エンティティを簡潔に参照するために使用します。
  SELECT * FROM Product の代わりに、このような記述方法を使います。

JPQL を記述する際、以下のルールを守る必要があります。

1. **エンティティ名を使用**
   Product(エンティティ名)を使用します。テーブル名(product)ではありません。
2. **フィールド名を使用**
   エンティティのフィールド名を指定します。列名(name や price)ではなく、Java のフィールド名をそのまま使います。
3. **SQL と似ているが異なる点もある**
   SQL とほぼ同じ構文ですが、SELECT 句や WHERE 句でテーブルではなくエンティティを扱うことを意識しましょう。

次は、コントローラーの全件検索用メソッドで、呼び出すメソッドを「 findAll」から、
「 findAllProducts」に変更します。

…/controller/ProductController.java

```
@GetMapping("/findAll")
        public String showProductList(Model model) {
                List<Product> list = prodRepo.findAllProducts();
                model.addAttribute("products", list);
                return "product/list";
        }
```

http://localhost:8080/product/findAll で動作確認を行うと、従来と同じように全件検索の結果が表示されることを確認できます。

JPQL の大きな利点は、エンティティ同士に外部参照(リレーション) が設定されている場合、JOIN 文を明示的に記述しなくても、関連するエンティティ(例えば Category) のデータも自動的に取得できる点です。

【 参考】大量データを扱う場合の注意点

大量のデータを扱う場合、関連する結合データが自動的に取得されることで、パフォーマンスに影響を与えることがあります。
このような場合、関連データのロード方法を変更することでパフォーマンスを改善できます。

以下のように@ManyToOne アノテーションのフェッチタイプを LAZY に変更します。

```
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "category_id")
private Category category;
```

結合データ(Category) は、明示的にアクセスされるまでロードされません。結果的に、無駄なデータのロードを防ぎ、クエリの実行速度が向上します。

WHERE 文の基本構文

これまでの章では、findById のようなリポジトリメソッドを使用し、検索条件をメソッドの引数で渡すことでデータを取得していました。

JPQL を使用した場合も同じように、検索条件を引数で渡してデータを検索できます。違いとしては、**@Query を使用して WHERE 文を直接記述する**点です。

以下は、指定された主キー(id)に一致する商品データを検索する JPQL の例です。

…/repository/ProductRepository.java

```
public interface ProductRepository extends JpaRepository<Product, Integer>{

    @Query("SELECT p FROM Product p")
     List<Product> findAllProducts();

    @Query("SELECT p FROM Product p WHERE p.id = :id")
     Product findProductById(int id);
```

- **WHERE p.id = :id**

  Product エンティティの **id フィールドが指定された値(:id)と一致**するデータを検索します。

- **:id**

  プレースホルダとして使用されるパラメータ。実行時に引数 id の値が挿入されます。

Java データベースプログラミングとの比較

理解のため、既存の知識である JDBC で主キー検索を行う場合と比較します。

```
String sql = "SELECT * FROM product WHERE id = ?";
PreparedStatement pstm = connection.prepareStatement(sql);
pstm.setInt(1, id);
```

- **JDBC でのプレースホルダ**

  SQL 文中の「 ?」がプレースホルダとして動的に値を受け取ります。
  パラメータの位置は、1 番目、2 番目と番号で指定します。

- **JPQL でのプレースホルダ**

  JPQL では、:id のような名前付きパラメータを使用します。番号ではなく、**パラメータ名と引数名を一致させることで紐付け**されるため、直感的で扱いやすいのが特徴です。

…/controller/ProductController.java

```
@GetMapping("/findById")
        public String showProductById(int id, Model model) {
                //Product product = prodRepo.findById(id).orElse(null);
                Product product = prodRepo.findProductById(id);
                model.addAttribute("product", product);
                return "product/detail";
        }
```

findById の代わりに、JPQL で定義したメソッドを呼び出します。

http://localhost:8080/product/ で ID 検索の動作確認をすると、同じ主キー検索ができるのが確認できます。

名前の LIKE 検索と価格の AND 検索の実装

商品名に特定のキーワードが含まれるデータを検索し、かつ、価格が指定された値と一致する商品のみを取得するという条件は、既存の JDBC 式では、以下のような記述が必要でした。

```
String sql = "SELECT * FROM product WHERE name LIKE ? AND price = ?";
PreparedStatement pstmt = conn.prepareStatement(sql));
pstmt.setString(1, "%" + name + "%"); // 名前の部分一致
pstmt.setInt(2, price); // 価格の条件
```

…/repository/ProductRepository.java

```
public interface ProductRepository extends JpaRepository<Product, Integer>{
        //他メソッドは省略

  @Query("SELECT p FROM Product p WHERE p.name LIKE %:name% AND p.price = :price")
    List<Product> findByNameContainingAndPrice(String name, int price);
}
```

JPQL を使う場合、name を囲む**ワイルドカード(%など)も一緒に記述**します。

…/controller/ProductController.java

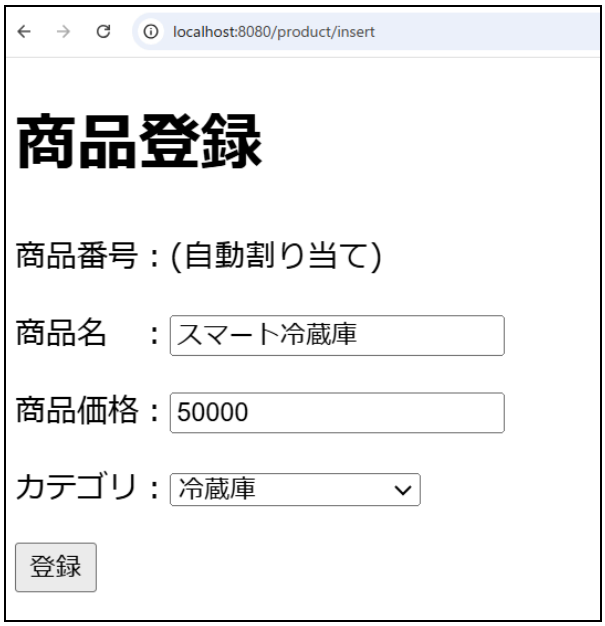
```
@GetMapping("/findByNameAndPrice")
public String showProductListByNameAndPrice(String name, int price, Model model)
{
    //List<Product> list = prodRepo.findByNameAndPrice(name, price);
    List<Product> list = prodRepo.findByNameContainingAndPrice(name, price);
        model.addAttribute("products", list);
        return "product/list";
}
```

コントローラーでは、既存の findByNameAndPrice メソッドの代わりに、JPQL で新たに定義した findByNameContainingAndPrice メソッドを呼び出すように修正します。

動作確認

http://localhost:8080/product/insert

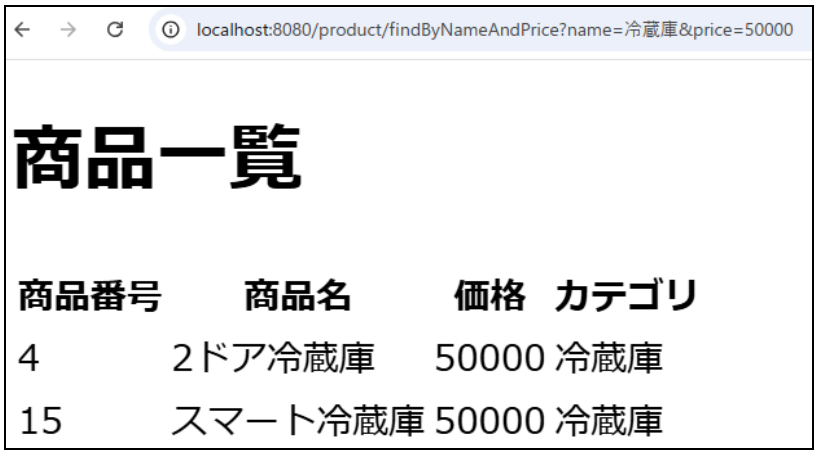
① スマート冷蔵庫と、50000、冷蔵庫を入力し、新たな商品を登録します。



② 以下の URL から、検索画面に遷移します。

http://localhost:8080/product/

③ 商品名+価格の検索欄に、「 冷蔵庫」と、「 50000」と検索します。



JPQL で定義した通り、「 冷蔵庫」というキーワード検索と、価格の AND 検索が行われていると正常です。

【参考】6.4.1 の例題を JPQL で作成する例

…/repository/ProductRepository.java

```
//【 参考】複雑な仕様の例題のサンプル①
        @Query("SELECT p FROM Product p LEFT JOIN p.category c " +
                        "WHERE p.name LIKE %:keyword% " +
                        "OR c.name LIKE %:keyword% " +
                        "OR p.description LIKE %:keyword% " +
                        "OR p.brand LIKE %:keyword% " +
                        "OR p.sku LIKE %:keyword%")
        List<Product> searchProductsByKeyword(String keyword);

//【 参考】複雑な仕様の例題のサンプル②
        @Query("SELECT p FROM Product p WHERE p.name LIKE %:keyword% " +
            "OR p.category.name LIKE %:keyword% " +
            "OR p.description LIKE %:keyword% " +
            "OR p.brand LIKE %:keyword% " +
            "OR p.sku LIKE %:keyword%")
        List<Product> searchProductsByKeyword(String keyword);
```

サンプル①は、より純粋な SQL 文に近い書き方で、**JOIN を利用した結合を明示的に記述**しています。

サンプル②は、**JOIN を利用せずに、p.category.name という書き方でエンティティの外部参照関係の定義を利用**する書き方です。

どちらも同じ結果になりますが、②の方がオブジェクト指向ですので、推奨します。

【参考】集計関数と GROUP BY の場合

Product と Category を対象に、JPQL を使った 集計関数(SUM, COUNT)と GROUP BY のリポジトリを作成する方法も参考として紹介します。

■ 実装するリポジトリメソッド

1. カテゴリごとの商品の合計価格を取得する (SUM(price))
2. カテゴリごとの商品数を取得する (COUNT(*))
3. 指定した価格以上の商品があるカテゴリ一覧を取得する (HAVING を使用)

【作業手順①】 DTO(集計結果を受け取るためのクラス)の作成

JPQL では DTO(データ転送オブジェクト) を作成して、結果をオブジェクトとして受け取るのが一般的です。

■カテゴリごとの合計価格を受け取る DTO

```
public class CategorySalesDTO {
    private String categoryName;
    private Long totalSales;

    public CategorySalesDTO(String categoryName, Long totalSales) {
        this.categoryName = categoryName;
        this.totalSales = totalSales;
    }

    public String getCategoryName() {
        return categoryName;
    }

    public Long getTotalSales() {
        return totalSales;
    }
}
```

■カテゴリごとの商品数を受け取る DTO

```
public class CategoryProductCountDTO {
    private String categoryName;
    private Long productCount;

    public CategoryProductCountDTO(String categoryName, Long
productCount) {
        this.categoryName = categoryName;
        this.productCount = productCount;
    }

    public String getCategoryName() {
        return categoryName;
    }

    public Long getProductCount() {
        return productCount;
    }
}
```

【 作業手順②】リポジトリの作成

```
@Repository
public interface ProductRepository extends JpaRepository<Product, Integer> {

    /* 1. カテゴリごとの商品の合計価格を取得 */
    @Query("SELECT new jp.co.example.dto.CategorySalesDTO(p.category.name,
SUM(p.price)) FROM Product p GROUP BY p.category.name")
    List<CategorySalesDTO> findTotalSalesByCategory();

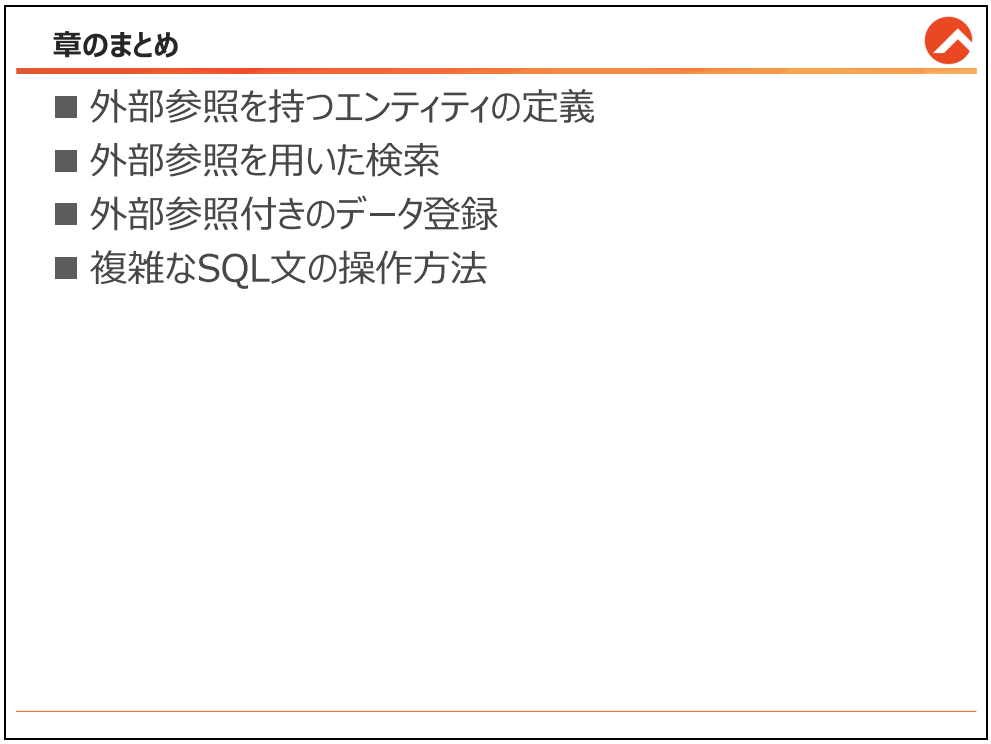
    /*  2. カテゴリごとの商品数を取得 */
    @Query("SELECT new
jp.co.example.dto.CategoryProductCountDTO(p.category.name, COUNT(p)) " +
           "FROM Product p GROUP BY p.category.name")
    List<CategoryProductCountDTO> findProductCountByCategory();

    /*  3. 指定した価格以上の商品があるカテゴリを取得( HAVING を使用)  */
    @Query("SELECT new jp.co.example.dto.CategorySalesDTO(p.category.name,
SUM(p.price)) " +
     "FROM Product p GROUP BY p.category.name HAVING SUM(p.price) >= :minSales")
    List<CategorySalesDTO> findCategoriesByMinSales(@Param("minSales") Long
minSales);
}
```

■まとめ

メソッド	目的	JPQL クエリ
findTotalSalesByCategory()	カテゴリごとの売上合計を取得	SUM(price) GROUP BY category.name
findProductCountByCategory()	カテゴリごとの商品数を取得	COUNT(*) GROUP BY category.name
findCategoriesByMinSales(Long minSales)	価格の合計が指定値以上のカテゴリを取得	HAVING SUM(price) >= minSales

章のまとめ



この章では、結合のための外部参照や JPQL について学習しました。